Laboration 2: Cellular Automata

Group Supermodels
Vootele Mets
Juan Rodriguez
Marco Malosti
Björk Lucas
Sofia Fernandes
David Weingut

May 5, 2026

UPPSALA
UNIVERSITET

Contents

1 Problem statement **2**

2 Game of Life **3**
2.1 Game of Life as a discrete dynamical system . . . 3
2.2 Encoding of Inputs . . . 4
2.3 Reading outputs . . . 4
2.3.1 Observation Window & Detection logic . . . 5
2.3.2 Synchorinization . . . 5
2.4 Numerical methods for Game of Life . . . 5
2.4.1 Computational domain . . . 5
2.4.2 Time evolution . . . 5
2.4.3 Initial conditions . . . 5
2.4.4 Logical gates . . . 5
2.5 Simulation Results . . . 7
2.5.1 NOT-gate . . . 7
2.5.2 AND-gate . . . 8
2.5.3 OR-gate . . . 10

3 Neuron Cellular Automaton **12**
3.1 Neuron Automaton Theory . . . 12
3.2 Neuron Cellular Automaton . . . 12
3.3 Neuron Automaton Results . . . 13

4 Greenberg-Hastings Cellular Automaton **15**
4.1 GHCA theory . . . 15
4.2 Methods . . . 15
4.2.1 Running the Code . . . 15
4.3 GHCA Results . . . 16

5 Interpretation **17**

6 Conclusion **18**

7 Use of LLM tools **19**
7.1 Game of Life . . . 19
7.2 Neuron Cellular Automaton . . . 19

1. Problem statement

In this lab we will be investigating classical examples of cellular automata. Cellular automata are ...

The goal of the lab is to ...

All of these tasks will give us an overview of the characteristics in cellular automata ...

2. Game of Life

2.1. Game of Life as a discrete dynamical system

Conway's Game of Life can be viewed as a discrete dynamical system on a rectangular lattice $X^{(t)} \in \{0, 1\}^{m \times n}$, where each entry $X_{ij}^{(t)}$ represents the state of the cell at site (i, j) at time t. The evolution of the system is determined by update rules that specify the state of each cell at the next time step, $X_{ij}^{(t+1)}$.

Definition 2.1 – Conway's rules [2]

1. **Survival**: a live cell stays alive if it has 2 or 3 live neighbors.
2. **Birth**: a dead cell becomes alive if it has exactly 3 live neighbors.
3. **Death by isolation**: a live cell dies if it has fewer than 2 live neighbors.
4. **Death by overcrowding**: a live cell dies if it has more than 3 live neighbors.
5. **Neighborhood**: the neighbors of a cell are the 8 surrounding cells.

Let $N_{ij}^{(t)}$ denote the degree of node $X_{ij}^{(t)}$ at time t. Since each cell has eight neighbors, the relevant directions are the cardinal and intercardinal directions. Self-interaction is excluded, so the evolution of a cell depends only on its neighborhood. The quantity $N_{ij}^{(t)}$ can be computed as

$$N_{ij}^{(t)} = \sum_{p=-1}^{1} \sum_{q=-1}^{1} K_{p+2,q+2} X_{i+p,j+q}^{(t)}, \tag{2.1}$$

where K is a 3×3 kernel with ones except the center entry that is equal to zero. Equivalently $N^{(t)} = K * X^{(t)}$, where $K * X^{(t)}$ denotes the convolution of the lattice $X^{(t)}$ with the kernel K.

The update rule can then be written as

$$X_{ij}^{(t+1)} = \begin{cases} 1, & \text{if } X_{ij}^{(t)} = 1 \text{ and } N_{ij}^{(t)} \in \{2, 3\}, \\ 1, & \text{if } X_{ij}^{(t)} = 0 \text{ and } N_{ij}^{(t)} = 3, \\ 0, & \text{otherwise.} \end{cases} \tag{2.2}$$

This shows that the update rule is a non-linear composition of a linear map:

$$X^{(t+1)} = \mathbb{1}\left(N^{(t)} = 3\right) + X^{(t)} \odot \mathbb{1}\left(N^{(t)} = 2\right), \tag{2.3}$$

We use dirichlet boundary conditons or zero padding to handle the adjacency of boundary nodes.

Logic gates in the Game of Life

In classical computation, the basic logic gates `AND`, `OR`, and `NOT` act on one or two binary input streams and produce a binary output stream. They implement a local transformation of information.

In the Game of Life, by contrast, computation is realized not by abstract Boolean operators but by spatially embedded dynamical processes on a lattice. The initial condition is a geometric configuration, and Conway's update rule evolves this configuration through time. The problem of building a logic gate is therefore a problem of geometric design and synchronization: one seeks an initial configuration $X^{(0)}$ that evolves, under the Game of Life rules, into a deterministic spatiotemporal process implementing `AND`, `OR`, or `NOT`.

This is achieved by using gliders as moving signals and glider guns as periodic sources of such signals [4, 5]. Glider trajectories function as wires, constraining the direction of information

flow; in the absence of interference, the signal propagates persistently along these paths. To represent binary information, the presence of a glider stream is interpreted as `1`, while its absence is interpreted as `0`. Eaters are used as stabilizing and filtering structures, suppressing selected streams and enabling controlled interactions between multiple inputs. By translating and positioning guns, eaters, and collision regions appropriately, one obtains deterministic and composable gate constructions [1], so that the output of one gate may be routed as the input of another without loss of logical meaning. computational fidelity.

2.2. Encoding of Inputs

Definition 2.2 – Encoding binary signals

Signal generation is achieved using the Gosper glider gun, which produces a periodic sequence of gliders with fixed phase and spacing. Through geometric transformations (rotations and reflections), identical guns can be oriented to produce signals in any required intercardinal direction. Each gun occupies a finite region of the lattice (for the canonical gun, 36×9 cells), but its function is not the static occupancy of this region; rather, it continuously emits a sequence of mobile patterns that carry information across the lattice. Consequently, the encoding is fundamentally dynamic and cannot be reduced to a fixed memory footprint.

1. **Signal 1** (`TRUE`): This is represented by the **prescence** of a **glider**. This signal propagates dynamically across a spatial lane across the lattice in fixed intercardinal directions.
2. **Signal 0** (`FALSE`): Defined as the **absence** of a **glider**. Its realization depends on the observation point. There are two basic cases:
   - **Blocking**: Physically *obstructing* a Gosper gun.
   - **Annihilation**: Using collision trajectories or *eaters* to remove a glider from the stream

Definition 2.3 – Signal Interaction Mechanics

Logic is performed through the interaction of these signals. Because gliders are sensitive to **synchronization**, the timing of these interactions is as important as their location.

1. **Glider-Glider Collision**: These collisions are different depending on their spatiotemporal initial configuration.
   - Requires: Antiparallel streams & precise phase sync.
   - Result: **Mutual Annihilation** (functioning as a switch) *or* **Fragmentation**. It also induces a space-dependent signal delay.
2. **Glider-Eater Interaction**: These interactions are *less* sensitive to synchronization.
   - Requires: Specific placement/orientation of the Eater.
   - Result: **Glider Annihilation/Absorpbtion** or **fragmentation**.

2.3. Reading outputs

Reading the state of a logic gate in the Game of Life is a measurement of **spatial-temporal presence**. The output of a gate is not solely a boolean value, but a **vector**. Since the gliders only propagate along intercardinal diagonals (NE, NW, SE, SW), the observation window must be placed precisely in the *firing lane* of the logic gate's exit path.

2.3.1. Observation Window & Detection logic

To detect a 1 signal (the presence of a glider), the observer must define a static bounding box of 4×4 cells at a specific coordinate (x, y) on the lattice. Because a glider is a dynamic pattern that fits within 4×4 area during its four-generation cycle. The output is considered to have the assigned `TRUE` value if the correct number of live cells within the 4×4, and the distribution are characteristic of the glider.

2.3.2. Synchorinization

If the observation window is checked at generation T, but the glider arrives at $T + 1$, the `READ` might detect a partial pattern (fragmentation) or an empty box, resulting in a false `0`. Strictly speaking, one could a priori calculate the Manhattan distance the glider travels to ensure the output is sampled only when the signal has had enough generations to reach the observation box.

2.4. Numerical methods for Game of Life

2.4.1. Computational domain

The computational domain is represented as a two-dimensional rectangular lattice $X \in \{0, 1\}^{n \times n}$, where each entry corresponds to one cell of the automaton.

2.4.2. Time evolution

The time evolution is implemented as a composition of a linear convolution operation and a point-wise non-linear update. The discrete convolution is computed using `scipy.ndimage.convolve`. A fixed 3×3 kernel is used to compute the Moore neighbourhood sum $N^{(t)} = K * X^{(t)}$.. The central entry of the kernel is zero, so the cell itself is excluded from the neighbour count. The update function allows both zero-padding boundaries, set by the parameter `mode="constant"`.

2.4.3. Initial conditions

Initial conditions are generated programmatically in `GoLogic.py`. The basic building blocks are the Gosper glider gun and the eater, both stored in Run Length Encoding (RLE). A custom parser converts each RLE string into a binary array using `RLE.py` and `add_pattern`. Before adding a pattern one can transform it appropriately with the `transform_pattern` function in `pattern_processing.py`. handle insertion and transformation before simulation.

In addition, the function `load_live_cells(filename, n)` loads external initial configurations from text files containing one live cell per line in the format `(x,y)`. The file is converted into an $n \times n$ binary array by setting `grid[y,x] = 1`. This supports the coordinate-list representation used for the initial gate configurations.

2.4.4. Logical gates

The logical gates `AND`, `OR`, and `NOT` are implemented by spatial arrangements of glider guns and eaters. Signals are encoded as glider streams emitted by Gosper guns. Boolean inputs are represented through the initial configuration: an input value of 1 leaves the corresponding signal unobstructed, whereas an input value of 0 places an eater on the signal path and suppresses propagation.

For the `AND` and `OR` gates, all four input combinations were simulated. While the `NOT` gate was tested for both possible input values. The output was read from a designated output lane: the

presence of a propagating glider stream was interpreted as logical 1, and its absence as logical 0.

This can be run as `python3 GameOfLife.py AND.txt`, adapting the text file for whatever is desired. For a different output, where no file is used, one can simply run the code as `python3 GameOfLife.py`.

2.5. Simulation Results

2.5.1. NOT-gate

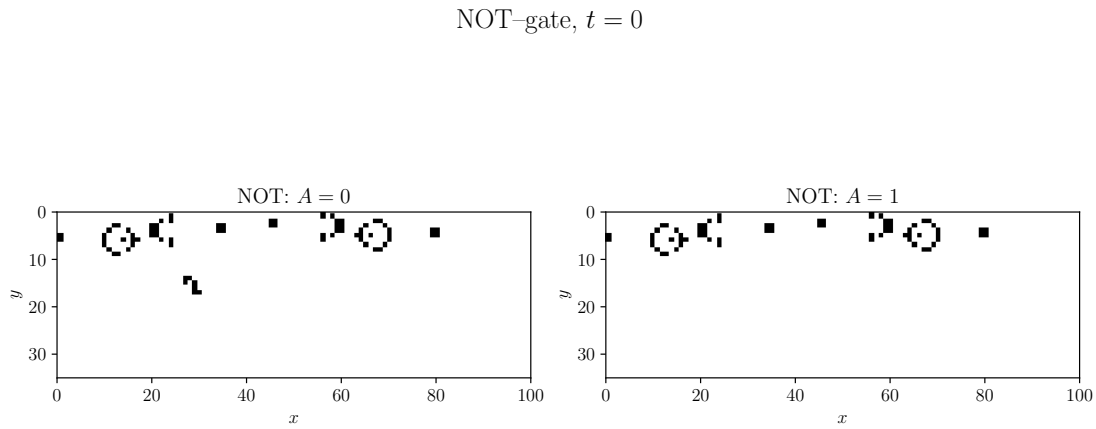

Figure 1. Two Gosper glider guns: the left gun provides a constant input stream, while the input channel injects gliders traveling in the south-east direction. No input is encoded by inserting an eater in the SE input stream. The output is read as gliders propagating in the south-west direction.

Initial conditions

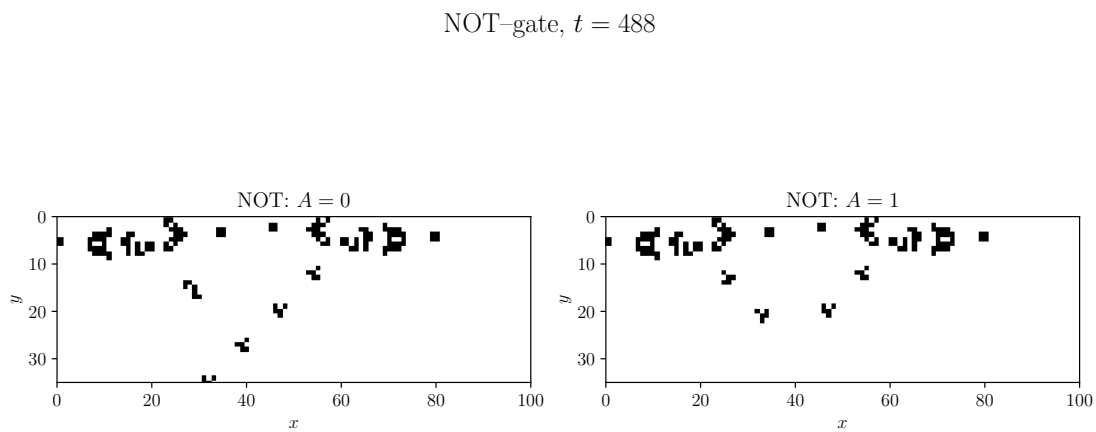

Figure 2. One input channel, A. The input glider moves diagonally from the top to the lower-right part of the grid. The output is observed after the interaction region, in the outgoing diagonal channel. The construction is arranged so that when $A = 0$ an output glider survives, while when $A = 1$ the input glider destroys or blocks the outgoing signal.

Transient state

96 **2.5.2. AND-gate**

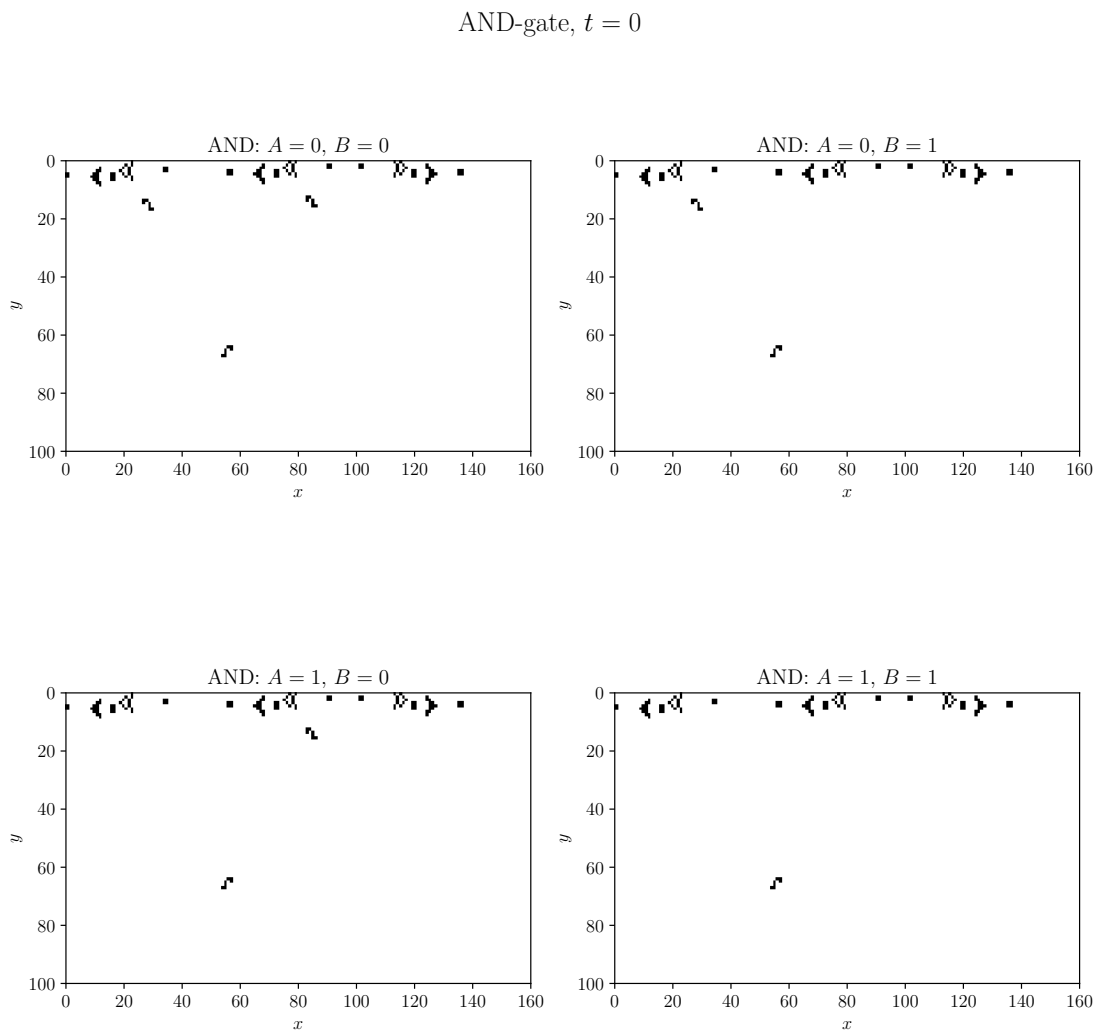

Figure 3. Three Gosper glider guns: the two leftmost generate south-east-directed input streams. An eater suppresses the south-west output unless both inputs are present. In the absence of inputs, additional eaters are placed along the collision paths to block spurious gliders. The output is read as a south-east-directed stream.

97 **Initial conditions**

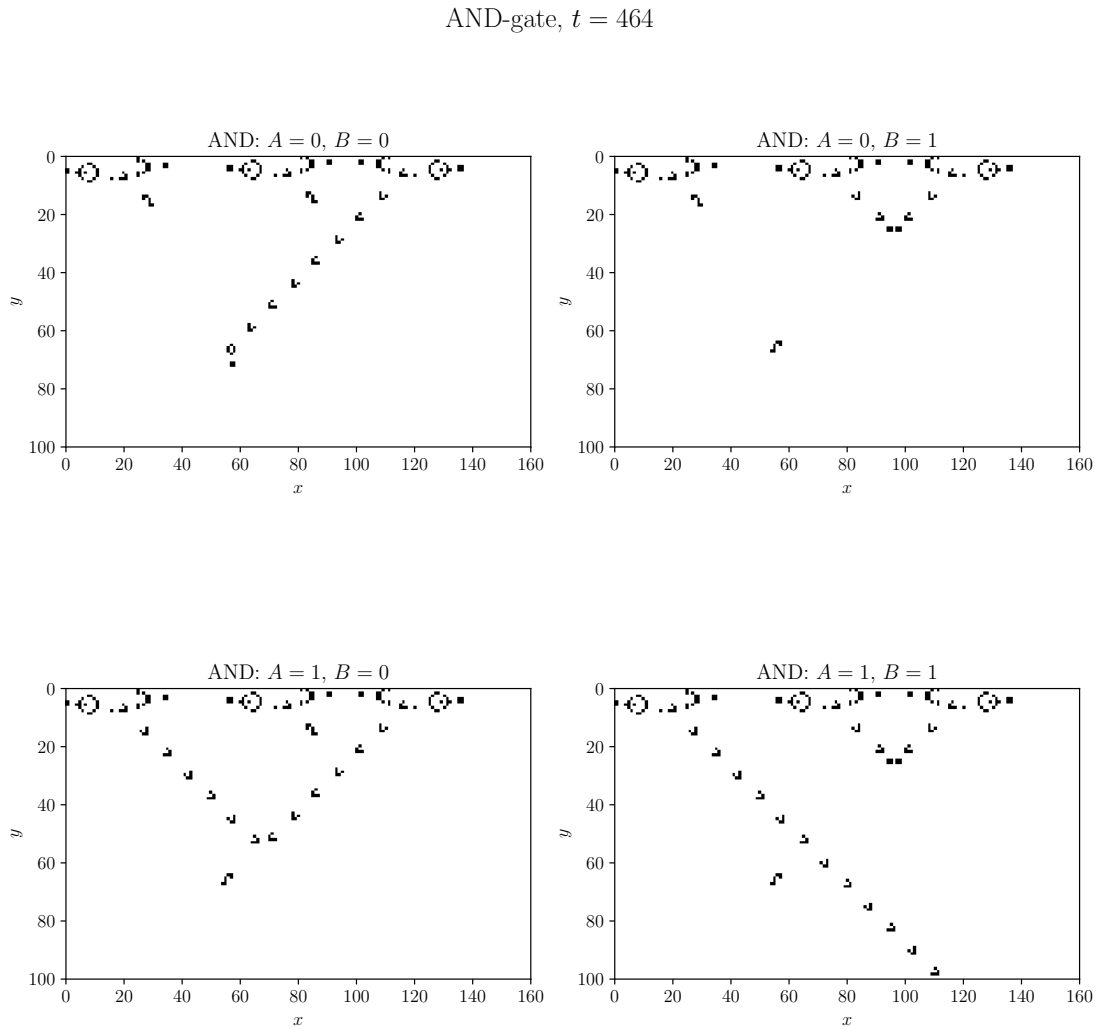

Figure 4. Two input channels are used, A and B. The input gliders move diagonally from the top of the grid towards the lower-right part of the grid, where they interact with the gate structure. The output is observed in the outgoing diagonal channel after this interaction. A glider appears in the output channel only when both input gliders are present, so the output is 1 only for $A = 1$ and $B = 1$.

98 **Transient state**

99 **2.5.3. OR-gate**

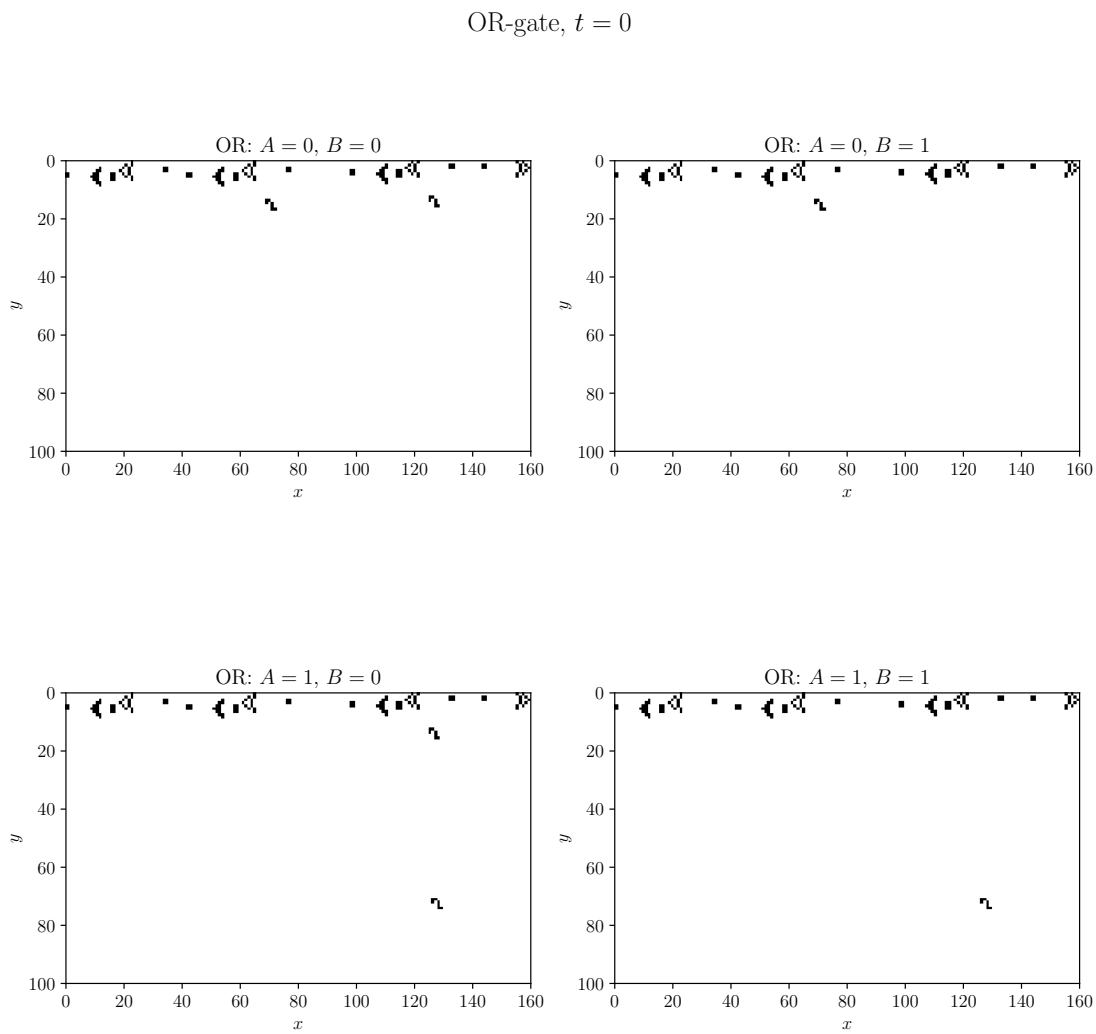

Figure 5. Four Gosper glider guns: the second and third generate south-east-directed input streams. Logical zeros are implemented by placing eaters along the input collision paths. The output is read as a south-east-directed stream.

100 **Initial conditions**

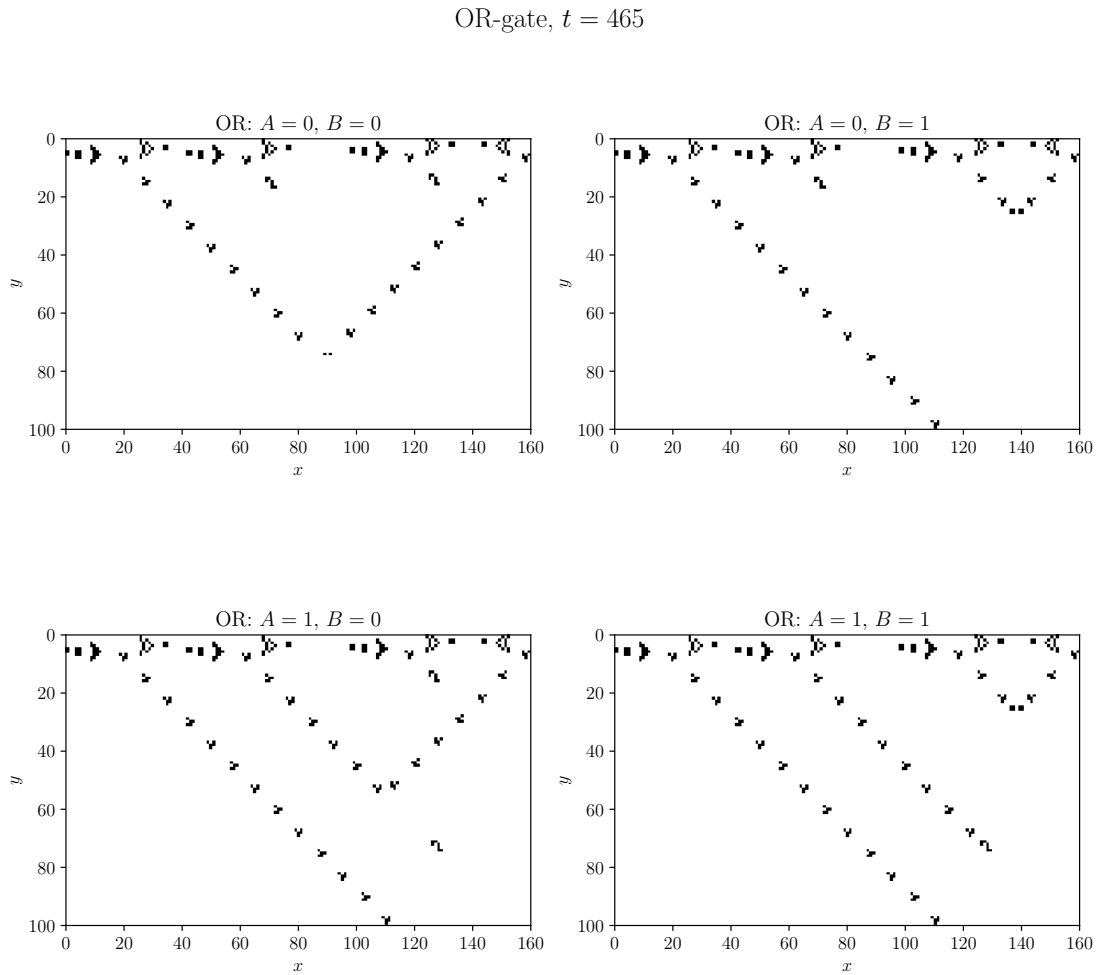

Figure 6. Input gliders also move diagonally from the top of the grid towards the lower-right part of the grid. The output is shown in the outgoing diagonal channel after the region of collision. In this case, the output channel contains a glider when, at least, one of the two input gliders is present.

101 **Transient state**

3. Neuron Cellular Automaton

3.1. Neuron Automaton Theory

Similar to the Game of Life, the neuron automaton is a two-dimensional cellular automaton. Each cell can be in one of three states: ”ready”, ”resting”, and ”firing”. The state of each cell at the next time step is determined by its current state of the cell and the states of its eight neighbors according to the following rules:

Definition 3.1 – Neuron cellular automata rules

1. **Firing**: a cell in the ”ready” state will transition to the ”firing” state in the next time step if it has at least two neighbors in the ”firing” state.
2. **Resting**: a cell in the ”firing” state will transition to the ”resting” state in the next time step.
3. **Ready**: a cell in the ”resting” state will transition to the ”ready” state in the next time step.

This kind of cellural automata simulates the behavior of neurons in a simplified way. The ready to firing to resting cycle mimics the action potential of a neuron, where a neuron fires when it receives enough stimulation from its neighbors and then goes through a refractory period before it can fire again. The interactions between cells can lead to complex patterns, including waves of activity and oscillations, which are reminiscent of neural activity in the brain.

3.2. Neuron Cellular Automaton

The script for the implementation of the neuron cellular automaton is `neuron.py`. The code is based on `gameOfLifeInteractiveToric.py`, which was available on the Studium page of the course.

Now, we will present how the code works. At first, the number of rows and columns of the grid as well the speed of the simulation are defined. Additionally, the rules text is added. This text is displayed when clicking the ”rules”-button while running the script. Next, we have implemented the new function `count_neighbors_by_state` that can calculate the number of neighbours in a certain state. This function is necessary as we will be working with three different states instead of two. The function takes the grid and the state that we want to determine the number of neighbours for as arguments. The neighbours for each gridpoint are found using `numpy.roll` to include the periodic boundary conditions. As the next step we have updated the `lifeStep` function. Now, we first create the grids of neighbours in states 1 and 2 and apply the rules afterwards. State 0 represents the resting neurons, state 1 the ready neurons and state 2 the firing neurons. The function then returns the updated grid.

In the constructor of the `GameOfLifeApp` class the visualisation is defined. This includes the buttons that are displayed and their actions. The time step for the continuous run is defined as well.

The following parameters are fixed in the code: the number of rows and columns is set to 30 each. For the random population of the grid, the probability for state 0 is 0.7, for state 1 the probability is 0.2 and 0.1 for state 2. Initially, all cells are in state 0 (resting). Clicking once sets them to state 1 and clicking another time to state 2.

3.3. Neuron Automaton Results

Using random initial conditions in the neuron automation usually led to either the whole system eventually dying out and being inert in the ”ready” condition or to infinitely forward moving gliders appearing. By experimenting several initial conditions were constructed that would lead to interesting developments of the system such as a periodic oscillator or a pattern that infinitely generates more moving patterns. Some interesting initial conditions are pictured below.

First a number of infinitely moving gliders were identified. On figure 7 are brought three examples. Those kinds of patterns move infinitely in the initial direction unless they collide with another pattern.

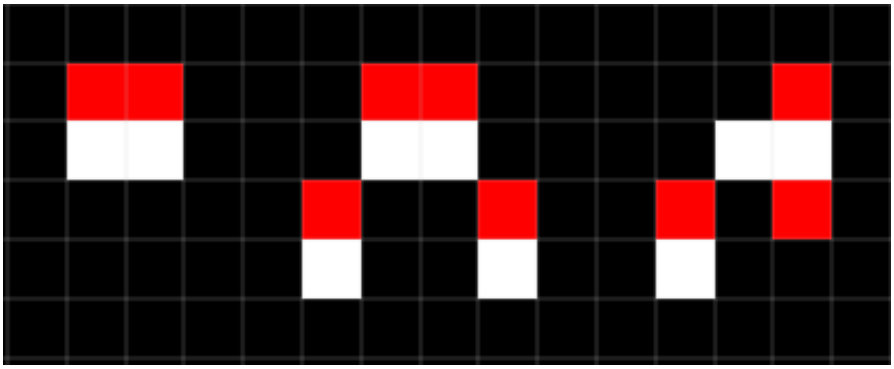

Figure 7. Three examples of infinitely moving patterns. Two patterns on the left move in the upward direction and the one on the right moves diagonally to right-left.

By having two simple gliders collide with eachother using initial conditions shown in figure 8 it is possible to create two new gliders that move at a right angle to the initial motion giving the impression of two bouncing gliders.

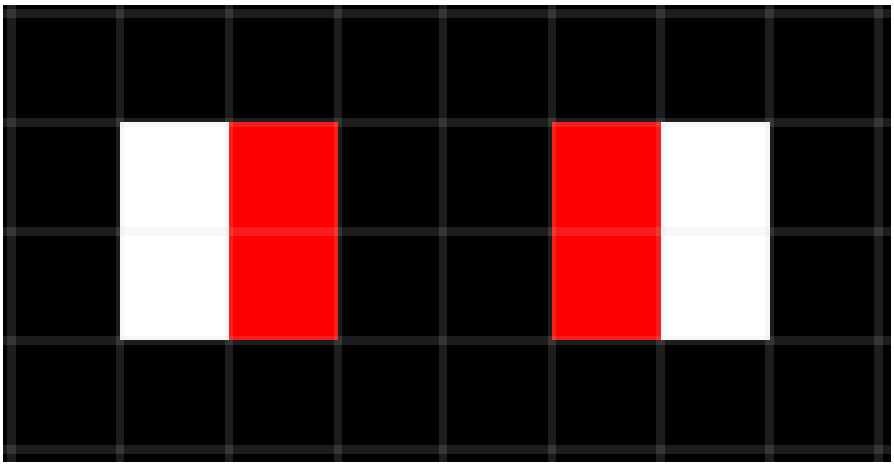

Figure 8. Initial condition that leads to two gliders bouncing in a 90 degree angle

It is also possible to set up a pattern that stais in the same location and oscillates infinitely. An oscillator with three distinct steps was identified that is pictured in figure 9

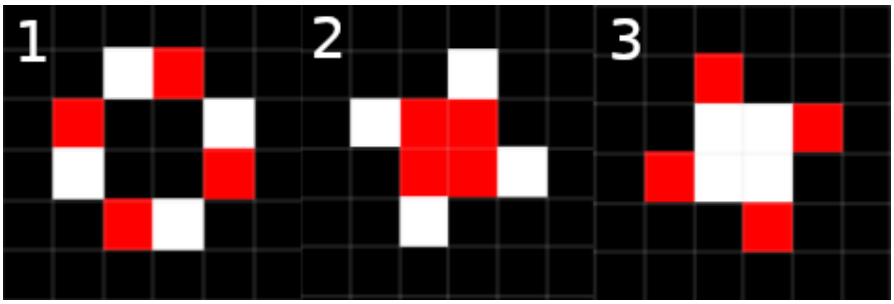

Figure 9. A three step oscillating pattern

On figure 10 is pictured a pattern that moves forward and periodically creates more moving patterns.

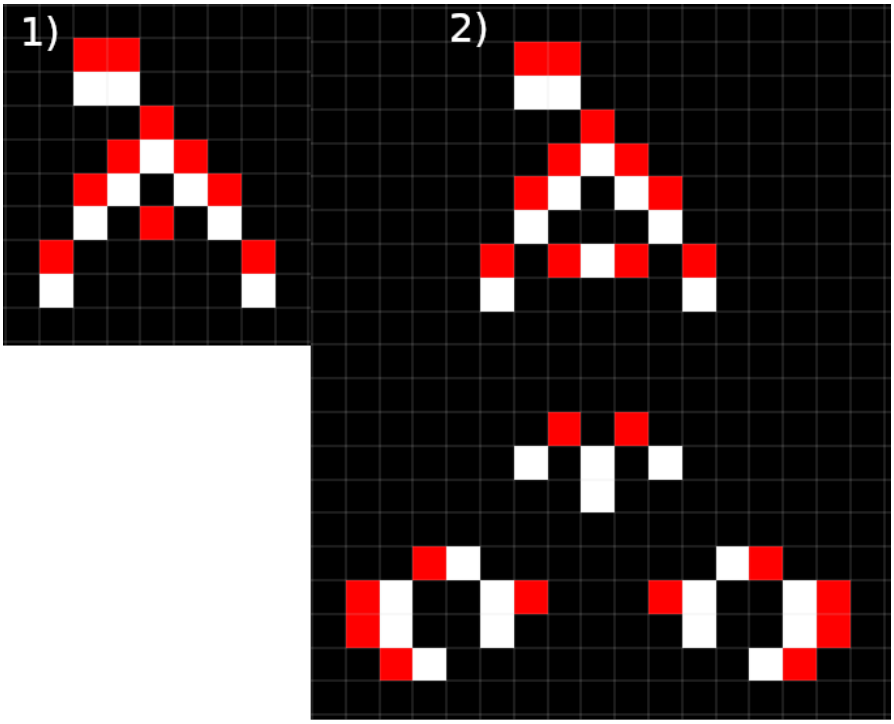

Figure 10. A pattern that while moving forward creates other moving patterns. 1) initial condition 2) system after some steps. the bottom two shapes are gliders moving to the left and right.

4. Greenberg-Hastings Cellular Automaton

4.1. GHCA theory

The Greenberg-Hastings Cellular Auomaton, GHCA for short, is a Cellular Automaton with three distinct categories of cell states. The first category consists of just a single state and is the `ready` state, assigned the numerical value 0 in our implementation. This state is the only one that can survive a step unchanged while for the others the value always gets incremented by one. The second category is `firing`. A total of e states are firing where e is an input parameter, this corresponds to the values 1 to e being the firing states in our model.

4.2. Methods

In general, the code was inspired by having a look at the interactive version that Jordi shared but was rewritten from the ground up to keep the complexity reduced for the actual task at hand. For representing the current state I chose to use a two-dimensional `numpy` array with integer values. The application of the von Neumann neighbourhood is done by four wrapping shifts in the two cardinal directions, for each once with a positive and once with a negative shift of one. The four shifted grids are assembled into a three-dimensional array to enable array operations on the entire neighbourhood at once. For each cell that is non-zero the value is incremented by one before applying the modulo operator to keep the values in the proper range. By the rules any ready cell is excited by at least one neighbour being in an excited state. This can be checked by testing each value if it is larger than zero but less than or equal to the excitation parameter. The any operation can be reduced over the third or stacking dimension. Now all cells that were previously ready and have any excited neighbour are set to the value one. For repeatedly stepping the model, the result of one step is used as the input to the next step. If at any point all cells are zero, the event loop is exited because the will be no further changes with the progression of time. Saving to and loading from file is done in a format that is similar to the format used for task 1 in the instructions. Each line consists of a tuple with three values, first the two coordinates as for indexing a `numpy` array and last the value of the cell at that coordinate. When loading the grid is initialized to be all zeros and overwrites all cells that are specified in the input. Drawing to a grid is done by using `matplotlib`'s `imshow` function and a colour map. Instead of reading from a configuration file the starting state can also be randomly generated. In that case 20 % of cells are set to random values in the interval 1 to N where N is the number of total states available for each cell.

For the aim to find the (eventual) periodicity of an initial configuration, the configuration is first read or an initial state generated. Afterwards, the initial state is saved to a list. Then, for each iteration we continue the automaton one step before checking if this new state is in this list. If it is, then we found a periodic state because the exact same state has occurred before. We exit the loop and optionally save the final state to a configuration file or an image and all members of the periodic orbit to individual images. If it is not in the list, we add the state to the list of seen states. The transient time can be found by getting the index of the previously seen state in the list. Any time steps before the recurrent state were transient while the time since the first occurrence is the period of the orbit.

4.2.1. Running the Code

The common functions for stepping the automaton, initializing a random grid, reading and writing configurations are in the file `ghca_common.py`. The code for the application fulfilling the requirements from the unnumbered list can be found in `ghca_advance.py` and can be configured with the following command-line arguments:

- `-s` Setting the size of the periodic grid
- `-n` Setting the number of steps before stopping
- `-e` Setting the number of excited states
- `-N` Setting the number of total states
- `--seed` Setting a random number seed for reproducibility of the random initialization
- `--show` Enables showing the current state of the grid in a `matplotlib` window
- `--delay` Setting the delay after each step before continuing the loop
- `-o` Setting the output file for writing the final state to a configuration file
- `--save-image` Setting the output file for an visual/image ersion of the final state
- the filename of the configuration file to read from as a positional argument

The code for determining the period can be found in the file `ghca_periodic.py`. The command line arguments are very similar with the following differences:

- no option to set the number of steps, it will run until it found a periodic orbit
- no delay parameter, for slowly inspecting the progression use the other program
- `--sequence-dir` to set a directory into which all members of the periodic orbit will be written as PNG files which can be used to create an animation

4.3. GHCA Results

The eventual periodicity of any input is clear from the finite state space. The only way to achieve a periodicity < 2 is if all cells are ready/in state 0 because that is the only state in which a cell does not change its value. This means that every initial configuration that does not die out becomes periodic with period ≥ 2. As the lecture already postulated[3, Lecture 9, slide 24], the period in our experimentation depends on the number of states and excited states. When $N \geq 5$ and $e \geq 2$ the period is exactly equal to the number of states. We did not prove this in a complete way, we only encountered no counter-example. For $e = 1$ I found periods $> N$. An example of this is the period 8 GIF uploaded as `431_442_453.gif` which is the periodic behaviour originating from the configuration shown in figure 11. This configuration is also a good way to elucidate the difference between the transient and periodic behaviour. While the initial configuration is very simple with only three cells being non-zero, the periodic configuration looks a lot more chaotic with different wavefronts. One member of the periodic orbit is shown in figure 12. The input configuration is supplied with the report as `periodic_N7_e1_size20.in` in the machine- and human-readable tuple format specifying coordinates and corresponding values. The values in the filename refer to the corresponding important simulation parameters number of states $N = 7$, number of excited states $e = 1$ and edge length size $= 20$.

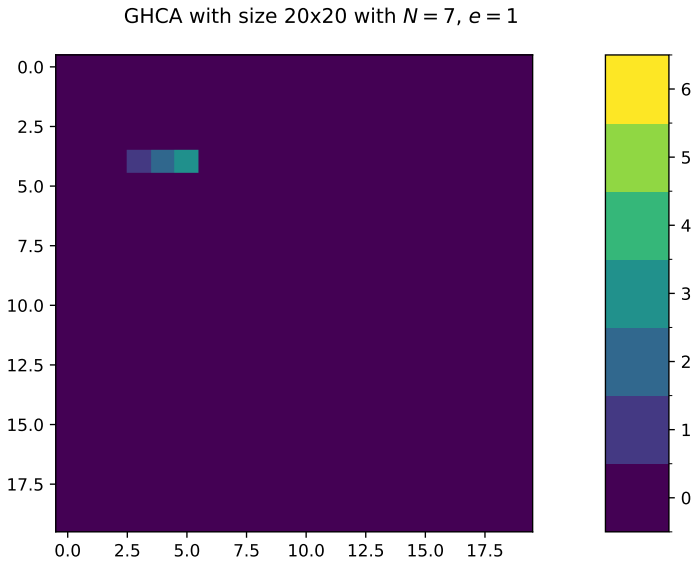

Figure 11. Simple Starting Configuration for Later Periodic Behaviour

5. Interpretation

INTERPRETATION GOES HERE.

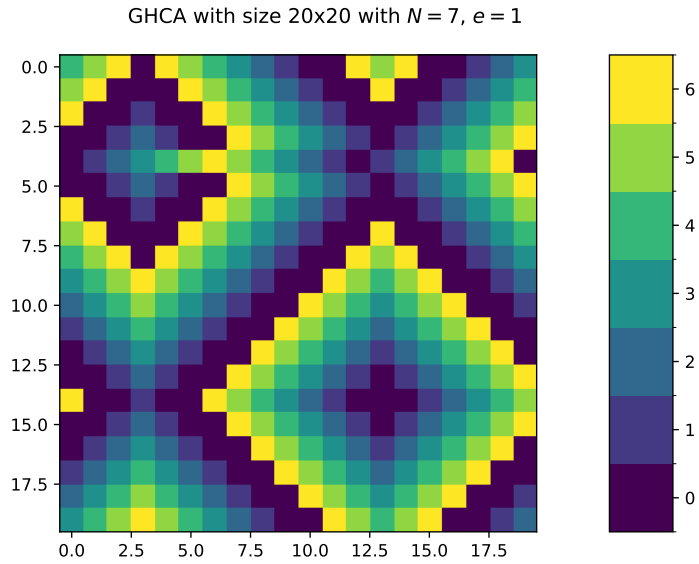

Figure 12. Member of Periodic Orbit Emerged from Simple Configuration

6. Conclusion

In Section (2.5) we presented snapshots illustrating the realization of logic gates within a computer simulation of the Game of Life. From this, one may conclude that a Boolean basis is effectively implemented within the system. Once the principles for composing these gates into larger circuits are understood, increasingly complex computational structures can be constructed.

The existence of logic gates in the Game of Life implies that they form a functionally complete basis for Boolean logic. Consequently, any Boolean function can be expressed as a finite composition of such gates. This property is fundamental for the construction of general computational devices. In particular, it is possible to design patterns that behave as a finite-state machine coupled to two counters. Such a system is computationally equivalent to a universal Turing machine

It follows that the Game of Life possesses the same theoretical computational power as any computer with unbounded memory and no time constraints; that is, it is Turing complete [6]. This demonstrates that complex dynamics can emerge solely from prescribed initial conditions and a simple update rule, forming a discrete dynamical system.

7. Use of LLM tools

7.1. Game of Life

RLE prompt: Create an rle loader to import standard Game of Life patterns directly. The output was a complete functional script `RLE.py` that parses strings like

```
24bo11b$22bobo11b$12b2o6b2...
```

, which encode binary patterns stored in a Wiki page.

Text Files prompt: Generate three text files containing the initial live-cell coordinates for an AND gate, an OR gate, and a NOT gate. The files should be called `AND.txt`, `OR.txt`, and `NOT.txt`. Each file should contain one live cell per line, written in the format `(x,y)` This was used to generate possible text files, so one can try running them with the code.

Debug interface prompt: Take the simple visualization of the precomputed Game of Life and add functionality so that space bar works as pause/play and left/right as previous/next frames and output the python file `GameOfLifeDebug.py`. This was used for quick prototyping of the engineering of the logic gates.

7.2. Neuron Cellular Automaton

The code in `neuron.py` is based on `gameOfLifeInteractiveToric.py` from the lecture. We used ChatGPT to modify the code. The prompt was ”I would like to change the rules of the game. I need to have three different states for each cell now and new rules, how can I implement this?” and additionally added the original code. This returned good instructions on where the code needs modification. Additionally, we looked up the documentation of the functions used by ChatGPT and made some modifications to its suggestions.

Files submitted

The following files have been submitted with this report:

1. `GameOfLife.py`
2. `GameOfLifeDebug.py`
3. `GoLogic.py`
4. `pattern_processing.py`
5. `RLE.py`
6. `NOT.txt`
7. `AND.txt`
8. `OR.txt`
9. `ghca_common.py`
10. `ghca_advance.py`
11. `ghca_periodic.py`
12. `431_442_453_periodic_N7_e1_size20.gif`
13. `periodic_N7_e1_size20.in`

References

[1] Alex Bellos. Game of life: Logic gates.

[2] Jordi-Lluís Figueras. Interactive conway's game of life board for classroom demonstrations, 2026.

[3] Jordi-Lluís Figueras. Lecture slides modelling complex systems, 2026.

[4] LifeWiki. Glider.

[5] LifeWiki. Glider gun.

[6] Wikipedia. Turing completeness.